Python Notes (Lecture 1)

Python : Basics

1. Programming

Programming is the process of giving a computer a set of instructions to perform a task. We write these instructions in a programming language such as Python.

A computer only understands **machine code** (0s and 1s), so a **translator** converts our code into machine code:

- **Compiler**: translates the whole program at once, then runs it.
- **Interpreter**: translates and runs the program line by line. Python uses an interpreter.

```
Your Code  →  Translator (Compiler / Interpreter)  →  Machine Code
```

---

2. What is Python?

- Simple and easy to learn: its syntax reads almost like English.
- Free and open source.
- High-level language: you don't manage memory or hardware details yourself.
- Developed by Guido van Rossum.
- Portable: the same program runs on Windows, macOS and Linux.

---

3. Our First Program

The `print()` function displays output on the screen.

```python
print("Hello World")
```

Output:

```
Hello World
```

Text must be written inside quotes. Separate multiple values with commas:

```
print("Python", "is", "fun")
print(10 + 5)
```

Output:

```
Python is fun
15
```

---

4. Python Character Set

These are the characters Python recognises:

- **Letters**: A to Z, a to z
- **Digits**: 0 to 9
- **Special symbols**: `+ - * / % = ( ) [ ] { } # : , .` etc.
- **Whitespaces**: blank space, tab, carriage return, newline, formfeed
- **Other characters**: Python can process all ASCII and Unicode characters as part of data or literals.

---

5. Variables

A variable is a **name given to a memory location** in a program. It stores a value we can use later.

```
name = "Shradha"
age = 23
price = 25.99

print(name)
print(age)
print(price)
```

Output:

```
Shradha
23
25.99
```

- The `=` symbol assigns the value on the right to the name on the left.
- Each value is stored in memory; the variable name points to that memory location.

- The value of a variable can change later in the program.

```
age = 23
age = 24      # value updated
print(age)
```

Output:

```
24
```

---

6. Rules for Identifiers

An **identifier** is the name given to a variable, function, etc.

- Can contain uppercase letters, lowercase letters, digits and underscore (`_`).
- Cannot start with a digit: `variable1` is valid, `1variable` is not.
- Cannot contain special symbols like `! # @ % $` or spaces.
- Can be of any length.
- Cannot be a keyword (such as `if` or `for`).
- Case-sensitive: `age` and `Age` are different.

```
myVariable = 10          # Valid
variable_1 = 20          # Valid
variable_for_print = 30  # Valid

# 1variable = 10         # Invalid: starts with a digit
# my-var = 10            # Invalid: special symbol
# my var = 10            # Invalid: contains a space
```

---

7. Data Types

Every value in Python has a type. The basic types are:

Type	Meaning	Example
`int`	Integer: whole number	`age = 23`
`float`	Decimal number	`pi = 3.14`
`str`	String: text in quotes	`name = "Shradha"`
`bool`	Boolean: True or False	`is_adult = True`
`NoneType`	None: no value	`result = None`

Use `type()` to check the type of a value:

```
age = 23
pi = 3.14
complex_num = 2 + 3j
A = True
name = "Shradha"
x = None

print(type(age))
print(type(pi))
print(type(complex_num))
print(type(A))
print(type(name))
print(type(x))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'complex'>
<class 'bool'>
<class 'str'>
<class 'NoneType'>
```

---

8. Keywords

Keywords are **reserved words** in Python. They have a special meaning and cannot be used as variable names.

`and`	`as`	`assert`	`break`	`class`
`continue`	`def`	`del`	`elif`	`else`
`except`	`finally`	`for`	`from`	`global`
`if`	`import`	`in`	`is`	`lambda`
`nonlocal`	`not`	`or`	`pass`	`raise`
`return`	`try`	`while`	`with`	`yield`
`True`	`False`	`None`		

Note: `True`, `False` and `None` start with a capital letter; all other keywords are lowercase.

---

9. Print Sum

Store two numbers in variables, add them and print the result:

```
a = 5
b = 10
sum = a + b
print("Sum is", sum)
```

Output:

```
Sum is 15
```

---

10. Comments

Comments are notes inside a program. Python **ignores** them when running the code.

A single-line comment begins with `#`:

```
# This is a single-line comment
print("Hello")   # comment after code
```

A multi-line comment is written between triple quotes:

```
"""
This is a
multi-line comment
"""
print("Comments make code easy to understand")
```

---

11. Types of Operators

An operator is a symbol that performs an operation on values (called **operands**).

a) Arithmetic Operators

`+` `-` `*` `/` `%` `**`

```
a, b = 10, 3

print(a + b)    # addition
print(a - b)    # subtraction
print(a * b)    # multiplication
print(a / b)    # division (always gives float)
print(a % b)    # remainder
print(a ** b)   # power: 10 to the power 3
```

Output:

```
13
7
30
3.3333333333333335
1
1000
```

b) Relational / Comparison Operators

`==` `!=` `>` `<` `>=` `<=`: these always give `True` or `False`.

```
a, b = 10, 3

print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Output:

```
False
True
True
False
True
False
```

c) Assignment Operators

`=` `+=` `-=` `*=` `/=` `%=` `**=`: shortcuts that update a variable.

```
num = 10
num += 5      # num = num + 5
print(num)
num *= 2      # num = num * 2
print(num)
num -= 10     # num = num - 10
print(num)
num **= 2     # num = num ** 2
print(num)
```

Output:

```
15
30
20
400
```

d) Logical Operators

`not` `and` `or`: used to combine conditions.

Operator	Result
`and`	`True` only if **both** conditions are True
`or`	`True` if **at least one** condition is True
`not`	Reverses the result

```
a, b = True, False

print(a and b)
print(a or b)
print(not a)
print((5 > 3) and (2 > 1))
```

Output:

```
False
True
False
True
```

---

12. Type Conversion

Python **automatically** converts one type to another when it is safe to do so (implicit conversion).

```
a, b = 1, 2.0
sum = a + b      # int is converted to float
print(sum)
```

Output:

```
3.0
```

But Python cannot automatically add a number and a string:

```
a, b = 1, "2"
sum = a + b      # Error!
```

Output:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

---

13. Type Casting

Type casting means **manually** converting a value from one type to another (explicit conversion).

```
a, b = 1, "2"
c = int(b)       # "2" becomes 2
sum = a + c
print(sum)
```

Output:

```
3
```

Common conversion functions:

Function	Description
`int(y)`	Converts y to an integer
`float(y)`	Converts y to a floating-point number
`str(y)`	Converts y to a string
`bool(y)`	Converts y to True or False
`complex(real, imag)`	Creates a complex number
`list(y)`	Converts y to a list
`tuple(y)`	Converts y to a tuple
`set(y)`	Converts y to a set
`dict(y)`	Creates a dictionary; y should be a sequence of (key, value) pairs
`ord(y)`	Converts a character to its integer (Unicode) value
`hex(y)`	Converts an integer to a hexadecimal string
`oct(y)`	Converts an integer to an octal string

```
print(int("25"))
print(float(5))
print(str(100))
print(ord("A"))
print(hex(255))
print(oct(8))
```

Output:

```
25
5.0
100
65
0xff
0o10
```

The value must be suitable for conversion: `int("hello")` gives an error.

---

14. Input in Python

The `input()` function is used to accept values from the user (using the keyboard).

Important: the result of `input()` is **always a string**.

a) input(): string

```
name = input("Enter your name: ")
print("Hello,", name)
print(type(name))
```

Output:

```
Enter your name: Aman
Hello, Aman
<class 'str'>
```

b) int(input()): integer

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)
```

Output:

```
Enter your age: 18
Next year you will be 19
```

c) float(input()): decimal number

```
price = float(input("Enter the price: "))
print("Price with 10% tax:", price * 1.1)
```

Output:

```
Enter the price: 50
Price with 10% tax: 55.00000000000001
```

Tip: small decimal errors like this are normal with floats. Use `round(value, 2)` to show 2 decimal places.

Let's Practice

Q1. Write a program to input 2 numbers and print their sum.

```python
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Sum =", a + b)
```

Output:

```
Enter first number: 12
Enter second number: 8
Sum = 20
```

Q2. Write a program to input the side of a square and print its area.

```python
side = float(input("Enter side of square: "))
area = side * side      # or side ** 2

print("Area =", area)
```

Output:

```
Enter side of square: 5
Area = 25.0
```

Q3. Write a program to input 2 floating-point numbers and print their average.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

average = (a + b) / 2
print("Average =", average)
```

Output:

```
Enter first number: 4.5
Enter second number: 7.5
Average = 6.0
```

Q4. Input 2 integers a and b. Print True if a is greater than or equal to b, otherwise print False.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

print(a >= b)
```

Output:

```
Enter a: 10
Enter b: 7
True
```

The comparison `a >= b` itself gives `True` or `False`, so we can print it directly.